

AI Practical Viva Questions & Answers

SPPU · TE Computer Engineering · Artificial Intelligence

All Important Topics • Grouped by Subject • Examiner-Focused

■ QUICK REFERENCE: Time & Space Complexities

Algorithm	Time Complexity	Space Complexity
BFS	$O(V+E)$	$O(V)$
DFS	$O(V+E)$	$O(V)$
Selection Sort	$O(n^2)$	$O(1)$
Bubble Sort	$O(n^2)$	$O(1)$
Prim's Algorithm	$O(V^2) / O(E \log V)$	$O(V)$
Kruskal's Algorithm	$O(E \log E)$	$O(V)$
Dijkstra's Algorithm	$O(V^2) / O(E \log V)$	$O(V)$
A* Algorithm	Depends on heuristic	$O(b^d)$
N-Queens (Backtracking)	$O(N!)$	$O(N)$
Graph Coloring	Exponential	$O(V)$
Minimax	$O(b^d)$	$O(bd)$
Alpha-Beta Pruning	$O(b^{(d/2)})$ best case	$O(bd)$

1. INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Q: What is Artificial Intelligence (AI)?

A: AI is a branch of computer science that develops systems capable of performing tasks that normally require human intelligence — such as learning, reasoning, decision-making, problem-solving, and understanding language.

Q: Who is the father of AI?

A: John McCarthy is known as the father of AI. He coined the term 'Artificial Intelligence' in 1956.

Q: What are the goals of AI?

A:

- Learning
- Reasoning
- Problem Solving
- Planning
- Natural Language Processing (NLP)
- Perception
- Robotics

Q: What are real-life applications of AI?

A:

- Expert Systems
- Robotics
- Self-driving cars
- Recommendation systems (Netflix, YouTube)
- Virtual assistants (Alexa, Siri)
- Medical diagnosis
- Game playing (Chess, Go)

Q: What is an Intelligent Agent?

A: An Intelligent Agent is an entity that perceives the environment using sensors and acts upon it using actuators to achieve defined goals.

Q: What is Rationality in AI?

A: Rationality means selecting the action that gives the best expected outcome based on available knowledge and goals.

Q: What is State Space Search?

A: A method of solving problems by exploring all possible states starting from an initial state and moving toward a goal state.

Q: Types of AI search — Informed vs Uninformed?

A: Uninformed (Blind) Search has no heuristic knowledge — examples: BFS, DFS. Informed (Heuristic) Search uses domain knowledge to guide search — examples: A*, Greedy Best-First Search.

2. BFS — BREADTH FIRST SEARCH

Q: What is BFS?

A: BFS (Breadth First Search) is a graph traversal algorithm that explores all neighbours of a node level by level before moving deeper.

Q: Which data structure is used in BFS and why?

A: Queue (FIFO). Nodes are explored level by level — the first node added is the first explored, which matches FIFO behaviour.

Q: What is the time and space complexity of BFS?

A: Time: $O(V+E)$ where V = vertices, E = edges. Space: $O(V)$ to store the visited array and queue.

Q: Why does BFS give the shortest path?

A: Because BFS explores nodes level by level. The first time the goal node is reached, it is guaranteed to be via the shortest path (minimum edges).

Q: What is the base condition in BFS?

A: The algorithm stops when the queue is empty (all nodes visited) or the goal node is found.

Q: What are applications of BFS?

A:

- Shortest path in unweighted graphs
- Social network friend suggestions
- Web crawlers
- GPS navigation (level-wise search)
- Peer-to-peer networks

Q: What happens if the visited array is not used in BFS?

A: Nodes may be revisited repeatedly, causing an infinite loop or extremely slow execution.

3. DFS — DEPTH FIRST SEARCH

Q: What is DFS?

A: DFS (Depth First Search) is a graph traversal algorithm that explores as deep as possible along a branch before backtracking.

Q: Which data structure is used in DFS and why?

A: Stack (LIFO). In iterative DFS, an explicit stack is used. In recursive DFS, the system call stack is used implicitly.

Q: What is the time and space complexity of DFS?

A: Time: $O(V+E)$. Space: $O(V)$ for the visited array and recursion stack.

Q: What is the base condition in recursive DFS?

A: The function stops (base case) when a node has no unvisited neighbours, and backtracks to the previous node.

Q: What is backtracking in DFS?

A: When DFS reaches a dead end (no unvisited neighbours), it reverses to the previous node and tries a different path. This is called backtracking.

Q: Why is DFS called uninformed search?

A: Because DFS has no heuristic knowledge about where the goal is — it blindly explores depth-first.

Q: Advantages of DFS?

A:

- Less memory than BFS
- Simple recursive implementation
- Useful for detecting cycles
- Good for backtracking problems like maze solving, N-Queens

Q: Disadvantages of DFS?

A:

- May get stuck in deep or infinite paths
- Does not guarantee shortest path
- Can loop infinitely without a visited array

Q: Applications of DFS?

A:

- Maze solving
- Topological sorting
- Cycle detection
- Connected components
- Backtracking puzzles

BFS vs DFS Comparison

Feature	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Traversal Order	Level by level	Depth first
Shortest Path	Yes (unweighted)	No guarantee
Memory Usage	More (stores all nodes at a level)	Less (stores one path)
When to use	Find shortest path	Explore all paths / backtrack
Time Complexity	$O(V+E)$	$O(V+E)$

4. A* ALGORITHM

Q: What is A* Algorithm?

A: A* is an informed (heuristic) search algorithm that finds the shortest/optimal path from a start node to a goal node using a cost function.

Q: What is the formula used in A*?

A: $f(n) = g(n) + h(n)$ where: $g(n)$ = actual cost from start to current node, $h(n)$ = heuristic estimated cost from current node to goal, $f(n)$ = total estimated cost.

Q: What is a heuristic function?

A: A heuristic function $h(n)$ estimates the cost from the current node to the goal. It guides the search towards the goal efficiently.

Q: Why is A* called an informed search?

A: Because it uses domain-specific heuristic knowledge ($h(n)$) to prioritize which nodes to explore next.

Q: What is an admissible heuristic?

A: A heuristic that never overestimates the actual cost to reach the goal. If $h(n) \leq$ actual cost, A* is guaranteed to find the optimal solution.

Q: What happens if the heuristic is bad (overestimates)?

A: A* may not find the optimal path. It could behave worse than uninformed search and miss the shortest route.

Q: Applications of A*?

A:

- GPS and map navigation
- Game pathfinding (NPC movement)
- Robotics path planning
- Maze solving
- Puzzle solving (8-Puzzle, 15-Puzzle)

8-Puzzle Problem

Q: What is the 8-Puzzle problem?

A: A 3x3 grid with 8 numbered tiles and one blank. The goal is to reach a target arrangement by sliding tiles into the blank space.

Q: What heuristic is used for 8-Puzzle in A*?

A: Two common heuristics: (1) Misplaced tiles — count of tiles not in their goal position. (2) Manhattan distance — sum of distances of each tile from its goal position. Manhattan distance is more accurate.

Q: Time and space complexity of 8-Puzzle?

A: Time: $O(b^d)$ where b = branching factor (~ 2.67), d = depth. Space: $O(b^d)$ to store open/closed lists. In worst case exponential.

Travelling Salesman Problem (TSP)

Q: What is the Travelling Salesman Problem?

A: Given N cities, find the shortest possible route that visits every city exactly once and returns to the starting city.

Q: Time and space complexity of TSP?

A: Exact solution: $O(N!)$ brute force or $O(2^N * N)$ with dynamic programming (Held-Karp). Space: $O(2^N * N)$. TSP is NP-Hard.

Q: How is A* used in TSP?

A: A* can be applied with a heuristic (e.g., minimum spanning tree cost as lower bound) to prune search space and find near-optimal tours.

5. GREEDY ALGORITHMS

Q: What is a Greedy Algorithm?

A: A greedy algorithm makes the locally optimal choice at each step, hoping this leads to a globally optimal solution.

Q: Why is it called 'greedy'?

A: Because it greedily picks the best immediate option without considering future consequences.

Q: When does greedy work correctly?

A: When the problem has the greedy choice property (local optimal leads to global optimal) and optimal substructure.

Q: Advantages of Greedy Algorithms?

A:

- Fast and simple to implement
- Efficient for many optimization problems
- Lower time complexity than exhaustive search

Q: Disadvantages?

A:

- Does not always guarantee a globally optimal solution
- Cannot backtrack once a choice is made

Selection Sort

Q: What is Selection Sort?

A: Selection Sort repeatedly finds the minimum element from the unsorted part and places it at the beginning of the sorted part.

Q: Time complexity of Selection Sort?

A: Best, Average, Worst: all $O(n^2)$ — because it always makes $n*(n-1)/2$ comparisons regardless of input.

Q: Space complexity of Selection Sort?

A: $O(1)$ — it is an in-place sorting algorithm.

Q: Is Selection Sort stable?

A: No. The relative order of equal elements may change during swaps.

Bubble Sort

Q: What is Bubble Sort?

A: Bubble Sort repeatedly compares and swaps adjacent elements if they are in wrong order, 'bubbling' the largest element to the end each pass.

Q: Time complexity of Bubble Sort?

A: Best: $O(n)$ with early termination. Average and Worst: $O(n^2)$.

Q: Space complexity of Bubble Sort?

A: $O(1)$ — in-place sort.

6. MINIMUM SPANNING TREE (MST)

Q: What is a Minimum Spanning Tree (MST)?

A: An MST is a subset of edges in a connected, undirected, weighted graph that connects all vertices with the minimum total edge weight and contains no cycles.

Q: Properties of MST?

A:

- Has exactly $V-1$ edges for V vertices
- No cycles
- Connects all vertices

- Minimum total weight

Prim's Algorithm

Q: What is Prim's Algorithm?

A: Prim's builds the MST by starting from any vertex and repeatedly adding the minimum weight edge that connects a visited vertex to an unvisited vertex.

Q: Approach of Prim's — Vertex-based or Edge-based?

A: Vertex-based. At each step it greedily picks the cheapest edge connecting the grown MST to a new vertex.

Q: Time complexity of Prim's Algorithm?

A: Using adjacency matrix: $O(V^2)$. Using binary heap + adjacency list: $O(E \log V)$.

Q: Space complexity of Prim's?

A: $O(V)$ for the key/parent arrays.

Q: Does Prim's work on directed graphs?

A: No. Prim's algorithm works only on undirected, weighted, connected graphs.

Q: Applications of Prim's / MST?

A:

- Network design (telephone, electrical)
- Road/highway planning
- Cluster analysis
- Approximation algorithms for TSP

Kruskal's Algorithm

Q: What is Kruskal's Algorithm?

A: Kruskal's builds the MST by sorting all edges by weight and adding the cheapest edge that does not form a cycle, until $V-1$ edges are added.

Q: Which data structure is used in Kruskal's and why?

A: Disjoint Set (Union-Find). It efficiently detects whether adding an edge creates a cycle using Find() and Union() operations.

Q: How does Kruskal's detect cycles?

A: Using Union-Find: if two vertices of an edge belong to the same set (same component), adding the edge would form a cycle, so it is skipped.

Q: Time complexity of Kruskal's?

A: $O(E \log E)$ — dominated by sorting edges. With efficient Union-Find: $O(E \log V)$.

Q: Space complexity of Kruskal's?

A: $O(V)$ for the Union-Find structure.

Prim's vs Kruskal's

Feature	Prim's	Kruskal's
Approach	Vertex-based (grow tree)	Edge-based (sort + select)
Data Structure	Priority Queue / Min-Heap	Sorted edges + Union-Find
Better for	Dense graphs	Sparse graphs
Time Complexity	$O(V^2)$ or $O(E \log V)$	$O(E \log E)$
Cycle Detection	Implicit (unvisited check)	Union-Find

7. DIJKSTRA'S ALGORITHM

Q: What is Dijkstra's Algorithm?

A: Dijkstra's algorithm finds the shortest path from a single source vertex to all other vertices in a weighted graph with non-negative edge weights.

Q: Which data structure does Dijkstra use?

A: Priority Queue (Min-Heap) to always extract the vertex with the smallest tentative distance.

Q: Time complexity of Dijkstra's?

A: Using adjacency matrix: $O(V^2)$. Using binary heap + adjacency list: $O(E \log V)$.

Q: Space complexity of Dijkstra's?

A: $O(V)$ for distance and visited arrays.

Q: Can Dijkstra handle negative edge weights?

A: No. Dijkstra fails with negative edges because once a node is marked visited its distance is assumed final — a later negative edge could give a shorter path.

Q: Difference between Dijkstra and BFS?

A: BFS works on unweighted graphs and uses a simple queue. Dijkstra works on weighted graphs and uses a priority queue to always pick the minimum-cost node.

Q: Difference between Dijkstra and Prim's?

A: Both use a greedy approach with a min-heap. Prim's builds a minimum spanning tree; Dijkstra finds shortest paths from a source. The key values updated differ.

8. CONSTRAINT SATISFACTION PROBLEM (CSP)

Q: What is a Constraint Satisfaction Problem (CSP)?

A: A CSP requires assigning values to variables from their domains such that all given constraints are satisfied. Components: Variables, Domains, Constraints.

Q: Examples of CSP?

A:

- N-Queens
- Sudoku
- Graph Coloring
- Scheduling
- Map Coloring

Q: What is Backtracking?

A: A recursive technique that builds a solution incrementally and abandons (backtracks) a path as soon as a constraint is violated, then tries an alternative.

Q: What is Branch and Bound?

A: An optimization technique that divides the problem into subproblems (branching) and eliminates branches whose best possible solution is worse than the known best (bounding).

Q: Difference between Backtracking and Branch & Bound?

A: Backtracking is used for feasibility — finding any valid solution. Branch & Bound is used for optimization — finding the best solution, pruning non-optimal branches.

N-Queens Problem

Q: What is the N-Queens problem?

A: Place N queens on an $N \times N$ chessboard so that no two queens attack each other — no two queens share the same row, column, or diagonal.

Q: What are the constraints in N-Queens?

A:

- No two queens in the same row
- No two queens in the same column
- No two queens on the same diagonal (both diagonals)

Q: How does backtracking work in N-Queens?

A: Place a queen row by row. For each row, try each column. If placing a queen violates any constraint, remove it (backtrack) and try the next column.

Q: Time complexity of N-Queens?

A: Worst case $O(N!)$ because in the worst case all permutations of N positions are tried. Backtracking prunes many branches so actual performance is better.

Q: Why is recursion used in N-Queens?

A: The problem is naturally divided row by row into smaller sub-problems — place a queen in row k , then recursively solve for row $k+1$.

Q: Real-life applications of N-Queens / CSP?

A:

- Resource allocation
- Job scheduling
- Puzzle solving
- Register allocation in compilers

Graph Coloring

Q: What is the Graph Coloring problem?

A: Assign colors to vertices of a graph such that no two adjacent (connected) vertices share the same color, using the minimum number of colors.

Q: Why is Graph Coloring a CSP?

A: Variables = vertices, Domain = available colors, Constraint = adjacent vertices must have different colors.

Q: What is the chromatic number?

A: The minimum number of colors needed to color a graph such that no adjacent vertices share a color.

Q: Applications of Graph Coloring?

A:

- Timetable / exam scheduling
- Register allocation in compilers
- Map coloring
- Frequency assignment in wireless networks

9. CHATBOT

Q: What is a Chatbot?

A: A chatbot is a software application that simulates human-like conversation using predefined rules, keywords, or AI/ML techniques to respond to user queries.

Q: Types of Chatbots?

A:

- Rule-based (keyword/pattern matching) — simple, fixed responses
- AI-based (ML/NLP) — learns from data, dynamic and context-aware

Q: What type of chatbot did you implement?

A: An elementary rule-based chatbot using keyword matching and predefined response templates.

Q: Why is a chatbot considered AI?

A: Because it simulates intelligent human interaction — it understands input (even if by pattern matching) and generates relevant responses, mimicking human communication.

Q: What is NLP and why is it used in chatbots?

A: Natural Language Processing (NLP) enables computers to understand, interpret, and generate human language. It allows chatbots to handle varied user inputs beyond exact keyword matching.

Q: Difference between rule-based and AI chatbot?

A: Rule-based: fixed patterns, no learning, simple to build, limited. AI-based: learns dynamically, understands context, uses ML/NLP, more powerful but complex.

Q: Limitations of a simple rule-based chatbot?

A:

- Cannot understand context or intent
- Limited to predefined responses
- No learning capability
- Fails for typos or unexpected phrasing

Q: Applications of Chatbots?

A:

- Customer support automation
- Banking and finance queries
- E-commerce product assistance
- Healthcare symptom checkers
- Virtual assistants (Siri, Alexa, Google Assistant)

10. ADVERSARIAL SEARCH & GAME THEORY

Q: What is adversarial search?

A: Search in a competitive environment where one player tries to maximize their score while the opponent tries to minimize it.

Q: What is the Minimax Algorithm?

A: Minimax is a decision algorithm for two-player games. The maximizer picks moves to maximize score; the minimizer picks moves to minimize the maximizer's score. It builds a complete game tree.

Q: What is Alpha-Beta Pruning?

A: An optimization of Minimax that prunes (skips) branches that cannot possibly affect the final decision. It produces the same result as Minimax but with fewer node evaluations.

Q: What are alpha and beta values?

A: Alpha = best score the maximizer is guaranteed so far. Beta = best score the minimizer is guaranteed so far. A branch is pruned when $\alpha \geq \beta$.

Q: Why is Alpha-Beta pruning important?

A: It reduces the number of nodes evaluated from $O(b^d)$ to $O(b^{d/2})$ in the best case, effectively doubling the search depth possible.

Q: What is Monte Carlo Tree Search (MCTS)?

A: A search algorithm that uses random simulations (playouts) to evaluate moves. It balances exploration of new moves and exploitation of known good moves.

11. KNOWLEDGE REPRESENTATION & LOGIC

Q: What is Knowledge Representation?

A: The method of encoding information about the world in a form that an AI system can store, reason about, and use to solve problems.

Q: Types of Knowledge Representation?

A:

- Logical representation (Propositional, FOL)
- Semantic networks
- Frames (structured data)
- Production rules (If-Then)

Q: What is Propositional Logic?

A: A formal logic using propositions (statements that are either TRUE or FALSE) connected by logical operators: AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$).

Q: What is First Order Logic (FOL)?

A: An extension of propositional logic that uses predicates, variables, constants, functions, and quantifiers ($\forall$ for all, $\exists$ there exists) to represent more complex knowledge.

Q: Difference between Propositional Logic and FOL?

A: Propositional Logic deals with simple true/false statements with no internal structure. FOL can represent objects, their properties, and relations between them — much more expressive.

Q: What are quantifiers in FOL?

A: $\forall$ (Universal quantifier) means 'for all'. $\exists$ (Existential quantifier) means 'there exists'. Example: $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ means 'All humans are mortal'.

Q: What is Unification in FOL?

A: Unification is the process of making two logical expressions identical by finding a substitution for variables. Example: $\text{Human}(x)$ unifies with $\text{Human}(\text{Ram})$ by substituting $x = \text{Ram}$.

Q: What is Forward Chaining?

A: A data-driven inference technique. Start from known facts, apply inference rules to generate new facts, and continue until the goal is reached.

Q: What is Backward Chaining?

A: A goal-driven inference technique. Start from the goal, work backward to find facts that support it. Used in Prolog and expert systems.

Q: What is the Wumpus World?

A: A grid-based AI environment (cave) used to demonstrate logical reasoning under uncertainty. The agent must find gold and avoid the Wumpus (monster) and pits using percepts like Stench, Breeze, and Glitter.

Q: What is Default Reasoning?

A: Reasoning using default assumptions when complete information is unavailable. Example: 'Birds can fly' is assumed unless an exception exists (e.g., penguins cannot fly).

12. PLANNING IN AI

Q: What is Planning in AI?

A: Planning is the process of generating a sequence of actions to transition from an initial state to a desired goal state.

Q: What is Classical Planning?

A: Planning in a fully observable, deterministic, static environment where all effects of actions are known in advance.

Q: What is Hierarchical Planning?

A: Breaking complex tasks into smaller, manageable subtasks (Hierarchical Task Networks). Simplifies planning and improves organization.

Q: What is Non-Deterministic Planning?

A: Planning in environments where actions may have multiple possible outcomes (uncertainty). The plan must handle all possible results.

Q: Why are heuristics important in planning?

A: Heuristics estimate how close the current state is to the goal, allowing planners to prioritize promising actions and dramatically reduce search time.

13. AI ETHICS & FUTURE

Q: What is AI Ethics?

A: The study of moral principles and responsible practices for designing, deploying, and using AI systems — ensuring they are fair, safe, transparent, and beneficial.

Q: Key ethical concerns in AI?

A:

- Bias and fairness in AI decisions
- Privacy and data security
- Job displacement due to automation
- Accountability for AI errors
- Misuse for surveillance or weapons

Q: What are the limitations of AI?

A:

- No emotions or common sense
- Requires large amounts of data
- High computational cost
- Cannot think creatively like humans
- Brittle — fails on inputs outside training distribution

Q: Future scope of AI?

A:

- Healthcare diagnostics and drug discovery
- Autonomous vehicles
- Smart cities and IoT
- Personalized education
- Advanced robotics and space exploration

14. MOST IMPORTANT LAST-MINUTE QUESTIONS

These are the questions most frequently asked by external examiners. Review these right before your exam.

Q: BFS vs DFS — key differences?

A: BFS uses Queue, gives shortest path, explores level by level. DFS uses Stack, explores depth first, uses less memory, used for backtracking.

Q: What data structures do BFS and DFS use?

A: BFS → Queue (FIFO). DFS → Stack (LIFO) or recursion call stack.

Q: What is a heuristic function?

A: $h(n)$ estimates cost from current node to goal. Used in A* as $f(n) = g(n) + h(n)$.

Q: Why is A* optimal?

A: A* is optimal when the heuristic is admissible (never overestimates actual cost).

Q: What is MST?

A: Minimum Spanning Tree — connects all V vertices with $V-1$ edges, minimum total weight, no cycles.

Q: Prim's vs Kruskal's — main difference?

A: Prim's is vertex-based (better for dense graphs). Kruskal's is edge-based (better for sparse graphs).

Q: Why does Dijkstra fail for negative edges?

A: Once a node is marked visited its distance is finalized. A negative edge found later could give a shorter path, which Dijkstra misses.

Q: What is the Greedy method?

A: Choose the locally optimal solution at each step. Fast but doesn't always give the global optimum.

Q: What is CSP?

A: Problem where values are assigned to variables satisfying constraints. Examples: N-Queens, Sudoku, Graph Coloring.

Q: What is Backtracking?

A: Try a solution incrementally; when a constraint is violated, undo (backtrack) and try the next option.

Q: What is Minimax?

A: Game tree algorithm — maximizer picks best move, minimizer picks worst move for opponent. Used in chess, tic-tac-toe.

Q: What is Alpha-Beta Pruning?

A: Optimization of Minimax that prunes branches that cannot influence the final result, reducing nodes evaluated.

Q: Propositional Logic vs FOL?

A: Propositional Logic: simple true/false. FOL: uses predicates, quantifiers, variables — more expressive.

Q: What is chatbot?

A: Software simulating human conversation using rules or AI. Rule-based uses keywords; AI-based uses ML/NLP.

All the best for your AI viva! You've got this. ■